

PROJET 1 : Module d'Authentification & Profil Utilisateur (e-joutia)

1. Objectif du projet

Développer le système d'accès sécurisé à l'application et la gestion du profil public d'un utilisateur, élément clé pour instaurer la confiance entre acheteurs et vendeurs.

2. Spécifications Fonctionnelles (Livrables UX/UI)

Écran de Connexion / Inscription : Formulaire complet avec saisie de l'email, mot de passe, nom d'utilisateur et ville de résidence.

Validation des entrées : Vérification en temps réel (email valide, mot de passe de 6 caractères minimum).

Écran Profil Public (Vue Vendeur) :

Affichage de la photo, du nom, de la localisation et de la date d'inscription.

Section "Avis" : Note moyenne (sur 5 étoiles) et liste des commentaires fictifs laissés par d'autres utilisateurs.

Badges de confiance (ex: "Profil vérifié", "Répond rapidement").

3. Contraintes Techniques & Architecture

Navigation : Utilisation d'une `Stack Navigation` (Switch automatique entre les écrans d'authentification et les écrans principaux après connexion).

Gestion des formulaires : Intégration de `Formik` ou `React Hook Form` couplé à `Yup` pour la validation.

Persistence : Stockage du token de session ou de l'état connecté via `AsyncStorage` pour éviter de se reconnecter à chaque ouverture.

PROJET 2 : Moteur de Recherche & Filtres Avancés

1. Objectif du projet

Créer une interface de recherche performante permettant aux utilisateurs de trouver rapidement des objets d'occasion spécifiques parmi un catalogue de données.

2. Spécifications Fonctionnelles (Livrables UX/UI)

Barre de Recherche Dynamique : Saisie textuelle avec historique des 3 dernières recherches enregistrées.

Affichage des Résultats : Présentation des produits sous forme de grille (2 colonnes) contenant : image, prix (€/DH), titre et distance en km.

Module de Filtrage (Bottom Sheet ou Écran Dédié) :

Filtre par Catégorie (Mode, Électronique, Maison, Véhicules, etc.).

Filtre par Prix (Slider ou champs min/max).

Filtre par État du produit (Neuf, Comme neuf, Bon état, Correct).

Tri (Le plus proche, Prix croissant/décroissant).

3. Contraintes Techniques & Architecture

Affichage de listes : Utilisation impérative de `FlatList` optimisée (props `windowSize`, `removeClippedSubviews` si nécessaire pour la performance).

Gestion d'état global : Centralisation des données du catalogue et des critères de recherche actifs via `Redux Toolkit` ou `Context API`.

Composants graphiques : Utilisation d'une librairie tierce pour le menu coulissant (ex: `@gorhom/bottom-sheet`) et le slider de prix.

PROJET 3 : Géolocalisation & Cartographie des Annonces

1. Objectif du projet

Exploiter les fonctionnalités matérielles du smartphone pour proposer une expérience d'achat géolocalisée de proximité, calquée sur le modèle de Wallapop Espagne.

2. Spécifications Fonctionnelles (Livrables UX/UI)

Écran de Configuration de la Zone : Permettre à l'utilisateur de définir sa position actuelle et de régler un rayon de recherche via un curseur (ex: 1km, 5km, 20km).

Vue Carte (Map View) :

Affichage d'une carte centrée sur la position de l'utilisateur.

Affichage de marqueurs (*Pins*) représentant l'emplacement approximatif des objets en vente.

Modale d'Aperçu Rapide : Un clic sur un marqueur ouvre une petite fiche en bas de l'écran affichant la photo du produit, le prix et la distance calculée (ex: "À 1.2 km de vous").

3. Contraintes Techniques & Architecture

Cartographie : Intégration de la librairie officielle `react-native-maps`.

Capteurs Natifs : Utilisation d' `expo-location` ou `react-native-geolocation-service` pour récupérer la latitude/longitude du téléphone.

Permissions : Gestion rigoureuse des demandes d'autorisation d'accès à la position de l'appareil (cas Refusé / Accepté).

PROJET 4 : Formulaire de Publication d'Annonce (Création)

1. Objectif du projet

Développer le tunnel de mise en vente d'un produit en combinant la saisie de données textuelles et la manipulation de fichiers multimédias natifs.

2. Spécifications Fonctionnelles (Livrables UX/UI)

Zone Multimédia : Un bouton permettant d'ouvrir l'appareil photo ou la galerie pour ajouter jusqu'à 5 photos du produit. Affichage des miniatures avec possibilité de supprimer une photo ajoutée par erreur.

Champs du Formulaire : Title (max 50 caractères), Description détaillée, Prix, Catégorie (Menu déroulant), État de l'objet.

Simulation de Soumission : Un bouton "Publier" qui affiche une animation de chargement puis un écran de succès confirmant la mise en ligne de l'annonce.

3. Contraintes Techniques & Architecture

Accès Caméra/Galerie : Utilisation de `expo-image-picker` ou `react-native-image-picker`.

Gestion des Médias : Manipulation correcte des URIs des images locales générées par le système.

Envoi de Données : Structuration de l'objet final sous forme de `FormData` (simulant un envoi réel vers une API contenant du texte et des fichiers binaires).

PROJET 5 : Messagerie Instantanée Intégrée (Chat Vendeur-Acheteur)

1. Objectif du projet

Créer le canal de communication en temps réel entre l'acheteur et le vendeur, incluant des fonctionnalités métiers spécifiques au commerce de seconde main.

2. Spécifications Fonctionnelles (Livrables UX/UI)

Écran Liste des Discussions : Liste des conversations actives affichant le nom de l'interlocuteur, l'avatar, la photo de l'objet concerné et le dernier message envoyé.

Fenêtre de Chat (Discussion) : Bulles de messages distinctes (Acheteur à droite, Vendeur à gauche) avec horodatage.

Module de Négociation (Fonctionnalité Wallapop) :
Présence d'un bouton *"Faire une offre"* en haut du chat.

L'acheteur propose un montant. L'offre apparaît graphiquement dans le fil de discussion sous forme de carte avec deux boutons pour le vendeur : "Accepter" / "Refuser".

3. Contraintes Techniques & Architecture

Interface Graphique : Utilisation recommandée de la bibliothèque `react-native-gifted-chat` pour accélérer le développement de l'UI du chat.

Simulateur Temps Réel / Backend : Utilisation au choix de `Firebase Firestore` (pour un vrai temps réel) ou une simulation robuste via un serveur local en WebSockets (`Node.js/Socket.io`).

Performance : Inversion de la liste pour charger les messages du bas vers le haut de manière fluide.

PROJET 6 : Système de Favoris & Liste d'Envies (Wishlist)

1. Objectif du projet

Concevoir un module de sauvegarde d'annonces axé sur l'expérience utilisateur, la réactivité de l'interface et la persistance locale des préférences.

2. Spécifications Fonctionnelles (Livrables UX/UI)

Interaction Micro-Animation : Sur n'importe quelle fiche produit de l'application, l'utilisateur peut cliquer sur une icône en forme de cœur. Le cœur doit changer de couleur (Vide -> Rouge rempli) avec un effet visuel dynamique.

Écran "Mes Favoris" : Onglet dédié qui répertorie toutes les annonces sauvegardées par l'utilisateur sous forme de liste.

Mise à jour en temps réel : Si l'utilisateur retire un objet de ses favoris depuis cet écran, l'élément doit disparaître immédiatement de la liste avec une animation de retrait fluide.

3. Contraintes Techniques & Architecture

Animations : Utilisation de `react-native-reanimated` ou de l'API `Animated` native pour créer l'effet de battement de cœur (*scale up/down*) lors du clic.

Persistance Globale : Utilisation d'un store global connecté à un système de persistance (ex: `redux-persist` ou Context API synchronisé avec un stockage local) pour que les favoris restent enregistrés même après fermeture complète de l'application.